SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

B-Tech Project

ForgeFlow: An AI-Native End-to-End Software Development Lifecycle Automation Platform

Group Number: Group – 04

Raghav Sonchhatra – PRN 23070122172

Sanidhya Awasthi – PRN 23070122192

Faheemuddin Sayyed – PRN 23070122196

Guided By:

Dr. Payal Kadam

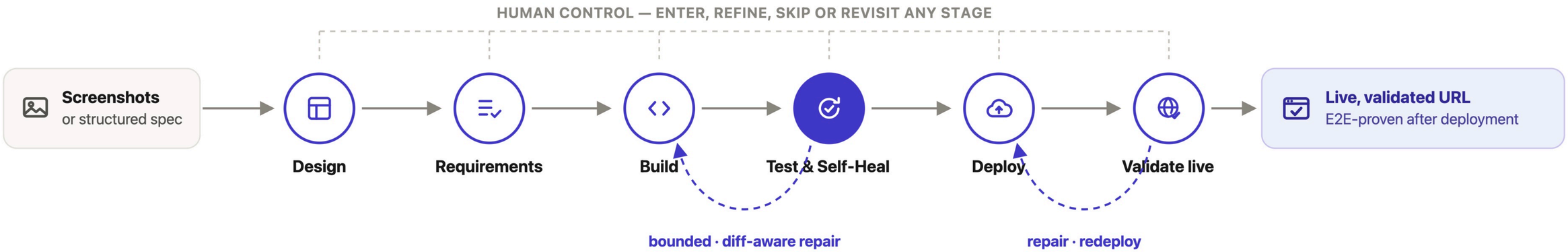
Professor

Computer Science

INTRODUCTION

ForgeFlow — An AI-Native, Human-in-the-Loop Platform for the Automated Web SDLC

Screenshots or a spec in → a tested, deployed website out.



PWA with an embedded browser IDE — Monaco editor and terminal built in; generated code stays visible and editable, not a black box.

No stage is mandatory. Enter anywhere, refine anything, skip anything, jump back at will — non-linearity is an invariant of the design.

Core technical contribution: a bounded, diff-aware self-healing repair loop — verification that terminates instead of thrashing.

Fixed generated stack — React + Express + MongoDB — gives higher first-pass success and makes rigorous verification tractable.

> “ **Existing tools generate apps. ForgeFlow generates apps it can *prove* work — and then proves it again on the live URL.**

The Adoption–Trust Gap Is Now the Central Problem in AI Software Engineering

Adoption is effectively universal

84% of developers already use — or plan to use — AI coding tools [1]

41–46% of all new code is now AI-generated; Gartner projects **60% by end-2026** [6]

$4.7B → $12.3B prompt-to-app market, 2026 → 2027 (≈38% CAGR) [6]

63% of prompt-to-app users are **non-developers** — no ability to debug what is generated [6]

Trust has collapsed in the same window



46% **actively distrust** the accuracy of AI output [1]

66% cite “almost right, but not quite” as their single biggest frustration [1]

45% say debugging AI-generated code takes **longer than writing it themselves** [1]

MOTIVATION (2/2)

And the Damage Is Measurable — Failures Survive QA and Reach Production

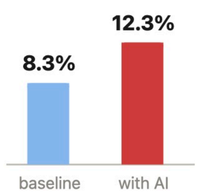
43%

of AI-generated changes still need manual debugging **in production — after passing QA and staging** [2]

0%

of surveyed engineering leaders were “very confident” AI fixes behave correctly once deployed [2]

88% needed 2–3 redeploy cycles per fix



Code duplication rose 8.3% → 12.3% across 211M lines analysed [3]

1.7×

more issues in AI-generated code than in human-written code [5] — and **45% of AI-generated code carries a security flaw** [4]

2,500%

Gartner-projected **increase in software defects by 2028** from citizen-developer prompt-to-app without quality controls [6]

> “ **Generation is solved. Verification is not.** That gap is exactly where we built.

STATUS OF CURRENT WORK

First Working Prototype — Complete End-to-End Pipeline Implemented

50 PLANNED PHASES

11 EPICS

The first prototype is implemented and functional, built agentically.
The hero path runs end to end:

Upload screenshots → Refine design → Generate app → Generate & run tests → Self-heal failures → Deploy → Validate live URL

SHIPPED SUBSYSTEMS

- **Control plane**
  FastAPI orchestrator, non-linear stage machine, conductor
- **PWA frontend**
  Workspace, stage navigator, conversational refinement
- **Web IDE**
  Monaco editor, xterm.js terminal, live code streaming
- **Execution**
  Isolated per-project Docker sandboxes, live preview
- **Design stage**
  Pluggable provider — Stitch MCP default, Figma alternative
- **Build stage**
  Agentic codegen on the fixed React / Express / MongoDB stack
- **Requirements → tests**
  Structured guided capture → unit + Playwright E2E suites
- CORE CONTRIBUTION
  Self-healing repair loop
  Diff-aware, bounded, escalating to a human on stall
- **Deploy**
  Infra analyzer + dual-mode (managed / BYO) — Vercel + Render
- **Live validation**
  E2E re-run against the deployed URL + repair path
- **Data**
  Per-project database isolation + in-app CRUD browser
- **Evaluation harness**
  Headless pipeline runner + metrics collection

> “ **This is not a mock-up. It is a working system — the review ahead is about hardening it, not building it.**

REFINED PROBLEM STATEMENT & SCOPE

Builders Can Generate and Deploy — They Cannot Reliably Verify

> **Current AI app-builders can generate and deploy web applications but cannot *reliably verify* them — leaving users in unbounded, costly bug-fix loops, with no validation of the live deployment and no beginner-safe path through cloud infrastructure.**

1

Verification, not just generation

Requirements compile into executable tests; failures drive a **bounded** repair loop.

2

Post-deployment truth

The same tests re-run against the live URL — passing locally is not evidence.

3

Infra-aware, dual-mode deploy

The system classifies what the app needs, then deploys managed **or** BYO-credentials.

4

Non-linear human control

Every stage optional, refinable and re-enterable — the human stays in charge.

OUT OF SCOPE (V1) × Mobile / native / desktop · × Multiple stacks · × Team collaboration · × Payments in generated apps · × Enterprise SSO / compliance

> **We deliberately narrowed to the web and one fixed stack — that constraint is what makes rigorous verification achievable.**

OBJECTIVES OF STUDY

Eight Objectives, One Thesis — Verified, Deployed, Human-Controlled Delivery

01

Non-linear orchestration

A human-in-the-loop model where no SDLC stage is mandatory and every artifact is versioned and re-enterable.

02

Agentic generation

Screenshots, prompts or user-supplied designs → a working full-stack web application.

03

Requirements → tests

Structured capture that compiles deterministically into unit and E2E suites traceable to acceptance criteria.

CORE

04

Bounded self-healing repair

Diff-aware patches with retry caps, no-progress detection and human escalation — developed **and validated**.

05

Infrastructure reasoning

Classify per-project deployment needs; execute dual-mode deployment with a live component topology.

06

Post-deployment closure

Validate the live URL after every deploy; repair and redeploy on failure.

07

Quantitative evaluation

First-pass vs post-repair success, repair convergence, cost and latency on a reproducible benchmark.

08

Isolation & safety

Per-project sandboxes and databases; untrusted generated code never executes in the control plane.

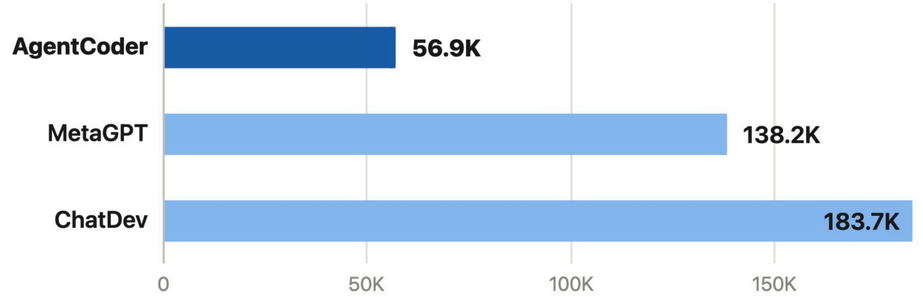
Agentic Code Generation Is Mature — and Converging

Multi-agent SDLC automation

ChatDev [8]	Models a virtual software company — designer / coder / tester roles across a waterfall lifecycle.
MetaGPT [9]	Encodes human SOPs into agent roles; reports 100% task completion and +5.4% absolute on MBPP via executive feedback.
AgentCoder [10]	**Fewer, better agents win:** 91.5% pass@1 (GPT-4) vs 86.8% SOTA — at roughly a third of the token cost.
SWE-agent / OpenHands [11][12]	Establish the agent–computer interface: shell, file and test-runner access as first-class tools.
MAGIS [13]	Targets repository-level issue resolution rather than greenfield generation.

Design2Code [21] (484 real webpages): multimodal models still lag on visual element recall and layout fidelity — validating our choice to keep design a **pluggable, human-refinable** stage, not a one-shot conversion.

Token overhead per task (thousands) [10]



Benchmark performance ≠ real-world reliability



Function-level puzzles are saturated; realistic repo-level tasks are not.
That gap is where products fail.

> “ The field converged on multi-agent generation. Nobody converged on knowing when to **stop fixing**.

Self-Repair Is Not a Silver Bullet — and Humans Stay in the Loop

Self-repair is not free

> *Once repair cost is accounted for, gains are "often modest, vary a lot … and are sometimes not present at all" — self-repair is* **bottlenecked by the model's own feedback quality**.
>
> Olausson et al., ICLR 2024 [14]

Human gap [14]	Even for the strongest models, self-repair lags far behind human-level debugging.
Self-Debugging [15]	Improves outcomes, but depends critically on the quality of execution feedback.

Direct design consequence: an unbounded repair loop is a cost sink, not a feature. ForgeFlow's retry caps, no-progress detection and escalation are a **response to this literature** — not an arbitrary limit.

Human-in-the-loop is the deployed industrial pattern

HULA [16]	Deployed inside Atlassian Jira — engineers keep control of planning and code at every step.
HULA follow-up [17]	Flags the two open challenges: the **high cost of correctness testing** and the variability of LLM-based evaluation.

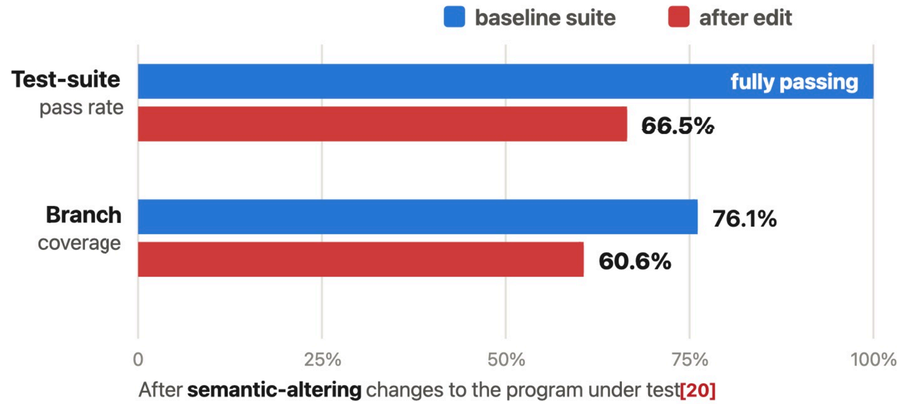
Test generation at scale — proven, but narrow

Schäfer et al. [18]	IEEE TSE empirical evaluation of LLM-based automated unit-test generation.
Meta (FSE) [19]	Industrial deployment of automated unit-test improvement at scale.
ICSE 2026 [22]	Program-analysis-guided iterative test generation — hybrid static/LLM pipelines.

The Strongest 2026 Evidence: LLM Tests Degrade Exactly When Code Changes

Haroon, Khan & Gulzar — arXiv, Mar 2026 [20] | 8 LLMs | 22,374 program variants | ≈346M tokens

baseline: 79.3% line / 76.1% branch coverage, fully passing suites



After **semantic-altering** changes to the program under test[20]

Even **semantic-preserving** *refactors drop the pass rate to 78.9%; test-suite match rate falls to 18.1% — models discard higher-coverage tests for weaker ones.*

>99% of the failing tests **pass on the original program** — "residual alignment": the model tests the code it *remembers*, not the code it was *given*.

> *"Computing a code diff and explicitly highlighting the modified regions could help guide the model's attention."*
> **— the authors' recommended fix** [20]

↓

This is **precisely ForgeFlow's diff-aware minimal repair context** — designed and built in Phase 29, independently corroborated by this study.

> **A March-2026 study, independently, tells you to do the exact thing we built.**

UPDATED LITERATURE SURVEY (3/3) — RESEARCH GAP

No Existing System Verifies, Deploys, Validates Live — and Stops

	ChatDev / MetaGPT [8] [9]	AgentCoder [10]	SWE-agent / OpenHands [11][12]	Lovable / Bolt / Replit (industry)	ForgeFlow
Multimodal design intake (screenshot → UI)	✗	✗	✗	◑	✓ pluggable
Full-stack app generation	✓	✗ function-level	◑ repo-level fixes	✓	✓
Requirements → executable tests (traceable)	◑	✓	✗	✗	✓ structured
Bounded repair — retry caps + stall detection	✗	✗	✗	✗	✓
Diff-aware minimal repair context	✗	✗	◑	✗	✓
Human escalation on non-convergence	✗	✗	◑	✗	✓
Automated cloud deployment	✗	✗	✗	✓ locked-in	✓ dual-mode
Post-deployment live validation	✗	✗	✗	✗	✓
Non-linear, skip-any-stage control	✗	✗	✗	◑	✓

✓ supported ◑ partial ✗ absent — highlighted rows: where the whole field says no

Research systems verify but do not deploy. Industry systems deploy but do not verify. **Neither validates the live artifact — and none bound the repair loop.**

PLAN OF NEXT WORK

Sixteen Weeks — Prove It, Break It, Harden It



Gap-finding & hardening · W1–4

- Failure-mode audit: adversarial specs, ambiguous requirements, broken deployments
- Sandbox security review — isolation, caps, secrets
- Resilience: provider fallbacks, crash recovery, resumable tasks

Repair-loop refinement · W3–7

- Tune iteration caps & stall thresholds against measured convergence
- Regression-guard on every patch
- **Ablation: diff-aware vs full-repo context — the headline result**

Evaluation at scale · W6–10

- Expand benchmark to 20–30 diverse app specifications
- Baselines: unbounded-repair & no-repair configurations
- Report convergence, cost per success, latency + variance

Production-worthiness · W8–14

- Observability: traces, cost dashboards, budget enforcement
- Multi-user robustness, quotas, rate limiting
- PWA finalisation + accessibility; seeded demo + fallbacks

Dissemination · W12–16

- Write-up for a software-engineering venue — the bounded loop + ablation is the paper
- Hackathon / expo submissions; demo runbook
- Open-source the eval harness + benchmark specs

> “ The build is done. The next five months are about **proving it, breaking it, and hardening it.**

CONCLUSION

Verification, Not Generation, Is the Frontier — and It Is Built

The industry has an **adoption–trust inversion**: 84% adoption [1] against 29% trust [1] — and 43% of AI code still failing in production after QA [2].

ForgeFlow addresses the failure that matters — **verification, not generation** — with a bounded, diff-aware, human-escalating repair loop.

A **complete first prototype is implemented**: 50 phases, 11 epics — screenshots → deployed and re-validated URL.

The design is **independently corroborated**: a March-2026 study of 22,374 program variants recommends diff-highlighted repair context after finding >99% residual alignment in LLM-generated tests [20].

The differentiator is defensible: research verifies without deploying; industry deploys without verifying. **ForgeFlow closes both loops — and stops.**

Next: hardening, adversarial gap-finding, scaled evaluation with ablations, and production-readiness.

> **“ Anyone can make an AI write an app. The hard part is knowing when it’s actually *finished* — and that is what we built.**

References

[1] Stack Overflow, "2025 Developer Survey: AI," Stack Overflow, 2025. [Online]. Available: https://survey.stackoverflow.co/2025/

[2] Lightrun, "2026 State of AI-Powered Engineering Report," Lightrun Ltd., 2026. [Online]. Available: https://venturebeat.com/technology/43-of-ai-generated-code-changes-need-debugging-in-production-survey-finds

[3] GitClear, "The Maintainability Gap: 2026 AI Code Quality Research," GitClear, 2026. [Online]. Available: https://www.gitclear.com/

[4] Veracode, "2025 GenAI Code Security Report," Veracode Inc., 2025. [Online]. Available: https://www.veracode.com/

[5] CodeRabbit, "AI-Generated Code Quality: An Analysis of 470 Real-World Pull Requests," CodeRabbit, 2026. [Online]. Available: https://byteiota.com/ai-code-quality-crisis-84-adoption-29-trust-in-2026/

[6] Hostinger, "Vibe Coding Statistics 2026: Adoption, Productivity, and Security Data," 2026, citing Gartner and Forrester forecasts. [Online]. Available: https://www.hostinger.com/blog/vibe-coding-statistics/

[7] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?," in *Proc. 12th Int. Conf. Learn. Represent. (ICLR)*, Vienna, Austria, May 2024.

[8] C. Qian *et al.*, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Bangkok, Thailand, Aug. 2024, pp. 15174–15186.

[9] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. 12th Int. Conf. Learn. Represent. (ICLR)*, Vienna, Austria, May 2024.

[10] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent code generation with iterative testing and optimisation," arXiv:2312.13010, 2024.

[11] J. Yang *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[12] X. Wang *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," in *Proc. 13th Int. Conf. Learn. Represent. (ICLR)*, 2025.

[13] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, and Y. Cheng, "MAGIS: LLM-based multi-agent framework for GitHub issue resolution," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 51963–51993.

[14] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?," in *Proc. 12th Int. Conf. Learn. Represent. (ICLR)*, Vienna, Austria, May 2024.

[15] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *Proc. 12th Int. Conf. Learn. Represent. (ICLR)*, Vienna, Austria, May 2024.

[16] W. Takerngsaksiri *et al.*, "Human-in-the-loop software development agents," arXiv:2411.12924, 2024.

[17] J. Pasuksmit *et al.*, "Human-in-the-loop software development agents: Challenges and future directions," arXiv:2506.11009, 2025.

[18] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 1, pp. 85–105, Jan. 2024.

[19] N. Alshahwan *et al.*, "Automated unit test improvement using large language models at Meta," in *Proc. 32nd ACM Int. Conf. Found. Softw. Eng. (FSE Companion)*, New York, NY, USA, 2024, pp. 185–196.

[20] S. Haroon, M. T. Khan, and M. A. Gulzar, "Evaluating LLM-based test generation under software evolution," arXiv:2603.23443, Mar. 2026.

[21] C. Si, Y. Zhang, R. Li, Z. Yang, R. Liu, and D. Yang, "Design2Code: Benchmarking multimodal code generation for automated front-end engineering," in *Proc. NAACL-HLT*, 2025.

[22] S. Gu, N. Nashid, and A. Mesbah, "LLM test generation via iterative hybrid program analysis," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, 2026.

[23] DemandSphere, "SWE-bench Verified — AI frontier model tracker," 2026. [Online]. Available: https://www.demandsphere.com/research/demandsphere-radar/ai-frontier-model-tracker/benchmarks/swe-bench/